Langage C

recto · le langage

mot-clé type intégré // commentaire «à remplir»

Types de base

1 · types · opérateurs

Entiers	`char short int`	1 · 2 · 4 octets (typique)
	`long`	8 (Linux 64) · 4 (Windows)
	`long long`	≥ 8 octets
	`unsigned int`	≥ 0, calcul modulo 2^n
Fixes	`int8_t uint32_t`	taille exacte · `<stdint.h>`
Réels	`float double`	4 · 8 octets (IEEE 754)
Booléen	`bool true false`	`<stdbool.h>` (mots-clés en C23) · 0 = faux
Tailles	`size_t`	non signé, type de `sizeof`
Vide	`void`	pas de valeur · `void *` générique

Littéraux

```
42   052   0x2A   0b101010     // déc, oct, hexa, bin (C23)
42u  42L   42UL   42LL         // suffixes de type
3.14  3.14f  1e-3              // double, float, 0.001
'A'  '\n'  '\0'  '\x41'        // caractères (= entiers)
"texte"                        // char[] terminé par '\0'
```

Conversions

```
7 / 2          // = 3   (division entière)
7 / 2.0        // = 3.5 (un réel suffit)
(double)a / b  // cast avant la division
(int)3.9       // = 3   (troncature vers 0)
-7 % 3         // = -1  (signe du dividende)
-1 < 1u        // FAUX : -1 converti en unsigned
```

Promotion : `char` et `short` deviennent `int` dans un calcul ; mélange signé / non signé ⇒ non signé.

Opérateurs par priorité

1	`() [] -> . x++ x--`	appel, indice, membre
2	`! ~ ++x -x *p &x`	unaires · droite → gauche
	`(type) sizeof`	cast, taille en octets
3	`* / %`	
4	`+ -`	
5	`<< >>`	décalages
6	`< <= > >=`	
7	`== !=`	
8	`&`	ET bit à bit
9	`^`	XOR bit à bit
10	`|`	OU bit à bit
11	`&&`	ET logique (court-circuit)
12	`||`	OU logique (court-circuit)
13	`c ? a : b`	condition · droite → gauche
14	`= += -= *= <<= &=`	affectations · droite → gauche
15	`a, b`	évalue a puis b, vaut b

⚠ **Piège :** `x & MASK == 0` se lit `x & (MASK == 0)` : parenthéser les opérations bit à bit.

Pièges classiques

- `if (x = 0)` affecte : écrire `if (x == 0)`
- `;` après `if (...)` ou `for (...)` : corps vide
- `switch` sans `break` : enchaîne les cas
- dépassement d'un `int` signé : comportement indéfini
- `scanf("%d", x)` : oubli du `&`
- chaîne sans place pour le `'\0'` final

Structures de contrôle

2 · contrôle · fonctions

```
if (cond) {                    // cond : 0 = faux, sinon vrai
    instructions
} else if (cond2) {
    instructions
} else {
    instructions
}

switch (expr_entière) {
    case 1:  instr; break;
    case 2:                    // sans break : continue
    case 3:  instr; break;     //   dans le case suivant
    default: instr;
}

for (int i = 0; i < n; i++) { ... }
while (cond) { ... }
do { ... } while (cond);       // au moins 1 passage
break;                         // sort de la boucle
continue;                      // passe au tour suivant
goto fin;  ...  fin: ;         // sortie d'erreur unique
```

Fonctions

```
int add(int a, int b);         // prototype (avant usage)
int add(int a, int b) {        // définition
    return a + b;
}
void swap(int *a, int *b) {    // modifier l'appelant :
    int t = *a;                //   passer l'adresse
    *a = *b;
    *b = t;
}
swap(&x, &y);
void f(void);                  // aucun paramètre
double moy(const int *t, size_t n);  // tableau + taille
static int aide(int x);        // privée au fichier .c
int main(int argc, char *argv[]) {   // argv[0] = nom
    return 0;                  // 0 = succès
}
```

Passage **toujours par valeur** : la fonction reçoit une copie. Un tableau passé en paramètre devient un pointeur sur son 1er élément (sa taille est perdue).

Portée et durée de vie

Locale	pile, détruite en fin de bloc, **non initialisée**
Globale	toute la durée du programme, initialisée à 0
`static` **loc.**	garde sa valeur entre deux appels
`static` **glob.**	visible dans ce fichier seulement
`extern`	déclare une globale définie ailleurs
`const`	lecture seule
`volatile`	relue en mémoire à chaque accès (registre, interruption)

```
int compteur(void) {
    static int n = 0;          // initialisé une seule fois
    return ++n;                // 1, 2, 3...
}
```

Pointeurs

3 · pointeurs · chaînes

```
int x = 5;
int *p = &x;                   // p = adresse de x
*p = 7;                        // x vaut 7
int **pp = &p;                 // pointeur de pointeur
p = NULL;                      // ne pointe sur rien
void *v = &x;                  // générique (pas de *v)
p + 1                          // avance de sizeof(int)
p[i]  ==  *(p + i)             // indice = arithmétique
q - p                          // écart en éléments
s->champ  ==  (*s).champ       // membre via pointeur
```

Lire une déclaration

`const int *p`	donnée constante, p modifiable
`int *const p`	p constant, donnée modifiable
`int *t[10]`	tableau de 10 pointeurs
`int (*t)[10]`	pointeur sur un tableau de 10
`int *f(int)`	fonction qui renvoie `int *`
`int (*f)(int)`	pointeur de fonction

Pointeurs de fonction

```
typedef int (*binop_t)(int, int);
binop_t op = add;              // nom = adresse
int r = op(2, 3);              // appel indirect
int cmp(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y); // -1, 0 ou 1
}
qsort(t, n, sizeof t[0], cmp);   // tri générique
```

Tableaux et chaînes

```
int t[5] = {1, 2};             // reste mis à 0
int u[] = {1, 2, 3};           // taille déduite : 3
int m[3][4];                   // m[i][j], ligne par ligne
size_t n = sizeof t / sizeof t[0];   // nb d'éléments
char s[] = "abc";              // 4 octets, '\0' compris
char *lit = "abc";             // littéral NON modifiable
void g(int m[][4], int rows); // 2D : colonnes fixées
```

⚠ **Piège :** `sizeof t` dans une fonction donne la taille du pointeur ; aucun contrôle des bornes : indices de 0 à n−1.

Bibliothèque standard

`<string.h>`	`strlen(s)`	longueur sans le `'\0'`
	`strcpy(d, s)`	copie (d assez grand)
	`strncpy(d, s, n)`	`'\0'` non garanti
	`strcat(d, s)`	ajoute s à la fin de d
	`strcmp(a, b)`	$< 0, 0, > 0$
	`strchr strstr`	→ pointeur ou NULL
	`memcpy memmove`	copie n octets ; recouvrement : move
	`memset(p, v, n)`	n octets mis à v
`<stdlib.h>`	`strtol(s, &e, b)`	chaîne → long, base b
	`rand() % n`	entier de 0 à n−1
	`srand(time(NULL))`	graine, une seule fois
`<math.h>`	`sqrt pow fabs`	lier avec `-lm`
`<ctype.h>`	`isdigit toupper`	test d'un caractère
`<assert.h>`	`assert(p != NULL)`	arrête si faux

⚠ **Piège :** `a == b` compare les adresses, pas le texte : `strcmp(a, b) == 0`.

Langage C

verso · données, mémoire et outils

mot-clé type intégré // commentaire «à remplir»

struct, union, enum

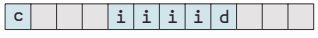
4 · données · macros

```c
struct point { int x, y; };
struct point a = {1, 2};
struct point b = {.y = 5};     // désignateur : x = 0
typedef struct node {
    int val;
    struct node *next;         // auto-référence
} node_t;
node_t n, *pn = &n;
n.val = 3;                     // accès direct
pn->val = 3;                   // via pointeur
b = a;                         // copie champ à champ
union mot {                    // champs superposés :
    uint32_t w;                //   même mémoire,
    uint8_t  o[4];             //   taille du plus grand
};
enum etat { IDLE, RUN = 5, STOP };   // 0, 5, 6
struct ctrl {                  // champs de bits
    unsigned en   : 1;
    unsigned mode : 3;
};
```

`a == b` interdit entre structures : comparer champ par champ.

Alignement et padding

```c
struct s1 { char c; int i; char d; };  // sizeof = 12
struct s2 { int i; char c; char d; };  // sizeof = 8
```



s1 : 12 octets dont 6 de bourrage

Chaque champ est aligné sur sa taille, la structure sur son plus grand champ : ranger les champs du plus grand au plus petit.

</> Préprocesseur

```c
#include <stdio.h>             // en-tête système
#include "module.h"            // en-tête du projet
#define N 100                  // constante (pas de ;)
#define SQ(x) ((x) * (x))      // tout parenthéser
#define STR(x) #x              // STR(a+b) -> "a+b"
#define CAT(a, b) a##b         // CAT(x, 1) -> x1
#ifdef DEBUG                   // gcc -DDEBUG
#define LOG(...) fprintf(stderr, __VA_ARGS__)
#else
#define LOG(...)               // disparaît
#endif
__FILE__ __LINE__ __func__     // position courante
```

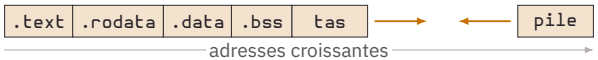
⚠ **Piège :** `SQ(i++)` incrémente deux fois : une macro recopie le texte.

Bits et registres

```c
x |=  (1u << n);               // met le bit n à 1
x &= ~(1u << n);               // met le bit n à 0
x ^=  (1u << n);               // inverse le bit n
if (x & (1u << n)) { ... }     // teste le bit n
v = (x >> 4) & 0xF;            // extrait les bits 7..4
#define GPIO_ODR (*(volatile uint32_t *)0x40020014)
GPIO_ODR |= 1u << 5;           // registre mappé
```

Zones mémoire

5 · mémoire



Pile	locales, paramètres ; libérée en fin de bloc ; quelques Mo
Tas	`malloc` / `free` : libéré à la main
.data	globales et `static` initialisées
.bss	globales et `static` non initialisées (mises à 0)
.rodata	constantes, littéraux `"..."`
.text	code machine

Allocation dynamique

```c
int *t = malloc(n * sizeof *t);   // non initialisé
if (t == NULL) {                  // toujours tester
    perror("malloc");
    exit(EXIT_FAILURE);
}
int *z = calloc(n, sizeof *z);    // mis à 0
int *tmp = realloc(t, 2 * n * sizeof *t);
if (tmp != NULL) t = tmp;         // échec : t intact
free(t);
t = NULL;                         // anti-pointeur pendant
```

⚠ **Piège :** un `free` par `malloc` ; jamais deux fois ; ne plus utiliser la zone après `free` ; jamais sur une variable de pile.

Matrice dynamique

```c
int **m = malloc(rows * sizeof *m);
for (int i = 0; i < rows; i++)
    m[i] = malloc(cols * sizeof **m);
m[i][j] = 0;                      // accès comme en 2D
for (int i = 0; i < rows; i++)    // libérer dans
    free(m[i]);                   //   l'ordre inverse
free(m);
```

Pointeur pendant

```c
int *mauvais(void) {
    int loc = 42;
    return &loc;                  // loc détruite au retour !
}
```

Exemple : liste chaînée

```c
node_t *push(node_t *head, int v) {
    node_t *n = malloc(sizeof *n);
    if (n == NULL) return head;
    n->val  = v;
    n->next = head;
    return n;                     // nouvelle tête
}
node_t *l = NULL;                 // liste vide
l = push(l, 1);
l = push(l, 2);                   // l : 2 -> 1
for (node_t *p = l; p != NULL; p = p->next)
    printf("%d ", p->val);
while (l != NULL) {               // libération
    node_t *suiv = l->next;
    free(l);
    l = suiv;
}
```

>_ E/S <stdio.h>

6 · E/S · compilation

`%d %u %ld %lld`	int, unsigned, long, long long
`%zu %p`	size_t, pointeur
`%x %X %o`	hexa, octal
`%f %e %g`	double (float promu)
`%c %s %%`	caractère, chaîne, %
`%5d %-5d %05d`	largeur, à gauche, zéros
`%.2f %8.3f`	précision · `%#x` : préfixe `0x`

```c
printf("x=%d y=%.2f\n", x, y);
fprintf(stderr, "erreur %d\n", code);
snprintf(buf, sizeof buf, "%d", x);   // borné
int nb = scanf("%d %lf", &i, &d); // nb lus ; %lf : double
int c = getchar();                // int, car EOF = -1
```

Fichiers

```c
FILE *f = fopen("data.txt", "r"); // "w" "a" "rb" "r+"
if (f == NULL) { perror("fopen"); return 1; }
char ligne[256];
while (fgets(ligne, sizeof ligne, f) != NULL)
    fputs(ligne, stdout);         // garde le '\n'
fprintf(f, "%d\n", x);            // écriture formatée
fread(buf, sizeof *buf, n, f);    // binaire
fclose(f);
```

>_ Compilation et débogage

Préproc.	`gcc -E`	`#include`, macros → `.i`
Compil.	`gcc -S`	C → assembleur `.s`
Assemb.	`gcc -c`	→ objet `.o`
Liens	`gcc`	`.o` + bibliothèques → exécutable

```
gcc -Wall -Wextra -std=c17 -g -o prog main.c mod.c
gcc -c mod.c                  # -> mod.o seul
gcc main.o mod.o -lm -o prog  # + libm (math.h)
```

`-O0 -O2`	optimisation
`-I dir -L dir`	chemins des .h, des .a / .so
`-DNOM=val`	`#define` depuis la ligne de commande
`-fsanitize=address`	débordements, fuites

module.h

```c
#ifndef MODULE_H              // garde d'inclusion
#define MODULE_H
extern int compteur;          // déclaration seule
int api(int x);               // prototypes publics
#endif
```

Makefile

```
CFLAGS = -Wall -Wextra -std=c17 -g
prog: main.o mod.o            # cible: dépendances
    $(CC) $^ -lm -o $@        # TAB devant la commande
%.o: %.c mod.h                # règle générique
    $(CC) $(CFLAGS) -c $<     # $< 1re dépendance
```

gdb et valgrind

`gdb ./prog`	programme compilé avec `-g`
`break f · run`	point d'arrêt dans f, lancer
`next · step`	ligne suivante · entrer dans l'appel
`print x · bt`	valeur · pile d'appels
`valgrind ./prog`	fuites : `--leak-check=full`